

Proofs using loop invariants

Introduction

So far, the main method we used to prove the correctness of algorithms in this course was induction. Proofs by induction are typically a good fit for problems based on recursion, because when we recur on a smaller subproblem, we are counting on the fact that the recursive call returns the expected result.

As we have seen, it is sometimes simpler or more efficient¹ to write recursive functions iteratively, that is, as a loop. In those cases (and even in some functions written using recursion) it isn't immediately obvious how to adapt the "recurrence on a smaller subproblem" argument in a proof by induction.

Proofs using a **Loop Invariant** are an alternative proof method that shares similarities with induction but is more intuitive for certain algorithms.

Definition 1: A Loop Invariant is a property of the program state that is true before the first iteration of a loop and that remains true after each iteration

Example 1: Maximum of an Array

Take the following code that finds the maximum element in a non-empty array.

```
FIND-MAX(A):  
  max_value <- A[0]  
  i <- 1  
  
  while i < length(A):  
    if A[i] > max_value:  
      max_value <- A[i]  
  
    i <- i + 1  
  
  return max_value
```

¹ In some cases, the iterative version is more efficient in practical implementation but has the same Asymptotic (Big O) time complexity. In other cases, without some additional bookkeeping between recursive calls, the recursive version could have drastically worse time complexity. This will be common in dynamic programming

Instructor: Rodrigo Canaan

“at the beginning of every iteration, the variable **max_value** contains the maximum element in $A[0..i - 1]$ ”

1. **Initialization.** Show that the invariant is true before the start of the loop

3. Termination. Show that the program terminates

4. Exit. Show that whenever the program terminates, the correct result is returned

Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

Exercise 1: Fill out the above proof with the justification of each step for FIND_MAX

Termination and loop Variants

The termination step is often formalized via a **Loop Variant**

Definition 2: a **Loop Variant** is a nonnegative integer-valued function of the program state that strictly decreases on every iteration in which the loop continues.

In the above case, a possible loop variant is the function $\text{length}(A) - i$ under $i < \text{length}(A)$. Because i increases by 1 every iteration, then (as long as this variant is positive to begin with) it must eventually reach zero, at which point the condition $i < \text{length}(A)$ no longer holds and the loop terminates.

Termination may seem obvious when the loop is simply scanning the array left to right, but some functions require a more sophisticated variant.

Example 2: Iterative Binary Search

Consider the following iterative version of binary search. Assume A is sorted:

BINARY-SEARCH(A, x):

$lo \leftarrow 0$

$hi \leftarrow \text{length}(A) - 1$

 while $lo \leq hi$:

$mid \leftarrow lo + \text{floor}((hi - lo) / 2)$

 if $A[mid] = x$:

 return mid

 else if $A[mid] < x$:

$lo \leftarrow mid + 1$

 else:

$hi \leftarrow mid - 1$

 return NOT_FOUND

Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

We can prove its correctness as such:

Invariant: At the beginning and end of every iteration, if x occurs in A , then it occurs at some index $lo \leq i^* \leq hi$

Initialization: Before the first iteration, $lo = 0$ and $hi = \text{length}(A) - 1$, that is, the search range is the entire array itself. If x occurs in A at all, it must occur in that range.

Maintenance: Assume that the invariant holds at the beginning of some iteration. There are three cases:

1. $A[mid] = x$. The algorithm returns mid , which is an index of x in A
2. $A[mid] < x$. Because the list is sorted, x cannot occur between lo and mid . Therefore, if it occurs between lo and hi at the start, it must occur between $mid+1$ and hi . We set lo to $mid+1$ and the invariant is maintained
3. Analogous to case 2, but x may only occur between lo and $mid-1$

Therefore, if the loop continues after this iteration (cases 2 and 3) the invariant is maintained

Termination: Consider the loop variant $n = hi - lo + 1$. This is the number of elements in the viable range. At every iteration we remove at least one index (mid) from consideration, therefore n is strictly decreasing as long as $lo \leq hi$ (which is the condition to remain in the loop).

Exit: The algorithm has two termination conditions:

1. **(successful exit)** At some iteration, $A[mid] = x$. As already covered, the algorithm correctly returns mid .
2. **(unsuccessful exit)** Whenever the loop terminates, we must have $lo > hi$. The invariant guarantees that if x in A , it must occur in the range defined by these two indices, but here the range is empty. Therefore does not occur in A at all. The algorithm correctly returns NOT FOUND.

Relationship with Induction

You may have noticed that the maintenance step of this Binary Search proof was very similar to the inductive step of our previous proof by induction. In fact, the initialization and maintenance steps of a loop invariant proof can be seen as induction on the iteration number:

1. **Initialization** : at the start of the first, the invariant holds [**Base case**]
2. **Maintenance**: If the invariant holds at the start of some iteration k [**Inductive Hypothesis**], then it must also hold at the start of iteration $k+1$ [**Inductive step**]

Therefore, for all $n \geq 0$, if the algorithm reaches iteration n , the invariant holds

Informally, the main difference is that proofs via invariant typically follow the iteration number (where the first iteration possibly deals with some large input) while proofs by induction follow the size of the input: we establish a base case (for a small input) assume correctness of all smaller inputs to justify the recursive calls on smaller subproblems.

The termination and exit steps guarantee that the algorithm doesn't run forever and that, when it does terminate, it returns the correct result.² For a concrete example of why the termination and exit steps are needed, consider the following exercises:

Exercise 2: Create an incorrect modification of iterative binary search where the invariant is still initialized and maintained, but the program fails to terminate.

Exercise 3: Create an incorrect modification of iterative binary search where the invariant is initialized and maintained, the program terminates, but exits with the wrong result.

² Proofs by induction on the input size will often leave termination implicit. The base case trivially terminates, and if every problem with input of size k terminates, then our recursive calls on subproblems are guaranteed to terminate. Since we only do some (finite) amount of work around the recursion, problems with input size $k+1$ also terminate

Conclusion

Loop invariants are properties of the program state that are true before the start of a loop **[initialization]** and at each of its iterations **[maintenance]**. If we also can show that the program eventually terminates **[termination]**. The invariant gives us a guarantee about the state of the program, which we rely on to show that, for each possible termination point of the program, it will return a value that is correct **[exit]**.

The idea that “certain properties are maintained as a system progresses” or “certain properties are maintained under some accepted transformations” is very broad and useful across various domains. Some examples:

- **Loop invariants:** the state of the program preserves some properties across iterations of a loop
- **Proofs by induction:** a property that is preserved regardless of input size
- **Conservation laws in physics:** the total energy of a closed system is conserved
- **Determinants in linear algebra:** Matrices can represent linear transformations under a specific choice of basis. The fact that the determinant is preserved under a change of basis shows that it is a property of the linear transformation itself, not of its specific representation
- **Parity in number theory and puzzles**
 - Mutilated chessboard and Knight's Tour problem: After covering a pair of tiles with a domino piece, the difference in number of black and white tiles is preserved. Therefore, there is no way to tile a chessboard that had its opposite corners (of the same color) removed. Similarly, every two Knight's move returns to the same color, so there is no knight's tour from diagonally opposite ends of the board.
 - Rubik's cube: you can tell that certain positions of the cube are unsolvable based on the parity of flipped edges. These positions can only be achieved on a physical cube by forcefully removing and reassembling pieces in the wrong orientation.
 - Not directly related to parity, but in solving a cube, it is useful to identify sequences of moves that execute desired changes while leaving vast regions of the cube unchanged.
- **Music:** A pitch shifted up or down by an octave (or having its frequency doubled/halved) is perceived as the same note. Shifting every note in a melody by an interval (multiplying every frequency by a fixed ration) is perceived as a key change. These make it possible for people with mostly little overlap in vocal ranges to be perceived as singing “the same melody” (either by singing in different octaves or changing keys)
 - This is due to a combination of biological and cultural factors, but provide a nice example of day-to-day “invariance”